-
-
-
-


```
┌──────────────────────────────────────┐
│              Agent                   │
│  ┌─────────┐  ┌─────────┐  ┌─────────┐  │
│  │  Model  │  │  Tools  │  │Instruct│  │
│  │ (   )   │  │ (   )   │  │ (   )   │  │
│  └─────────┘  └─────────┘  └─────────┘  │
└──────────────────────────────────────┘
```


-
-
-

•
•
•
•

•
•
•
•

-
-
-
-

📚

```
Agent = LLM + Memory + Planning + Tool Use
```


```
→ LLM     →          →              →
      ↑                                ↓
      └──────────                ──────────┘
```


-
-
-
-

-
-
-
-

-
-
-

	✅	❌	❌
	✅	❌	❌
	❌	✅	✅

		→

```
→    1(    ) →    2(    ) →    3(    ) →
```

-
-
-

→ →

```
→       →    A (        →      )
          →    B (        →      )
          →    C (        →        )
```

-
-
-

```
→      A ─┐
 →      B ─┼─→
 →      C ─┘
```

```
→      1 ─┐
 →      2 ─┼─→
 →      3 ─┘
```

```
Orchestrator:        →
              ├→ Worker1:      A
              ├→ Worker2:      B
              └→ Worker3:      C
             ←
```

→ → →

-
-
-

→ → →

-
-
-
-

①

- _____
- _____
- _____

-
-

```
Thought:
Action: search_web("2025                ")
Observation:             ...
Thought:
Action: search_web("2025                   ")
Observation: ...
Final Answer: ...
```

```
Question
  ↓
Thought → Action → Observation
  ↑                       ↓
  └──────          ──────┘


                Final Answer
```

```
You are an agent that can use tools.
When solving tasks, follow this format:
Thought: think step by step
Action: one tool name
Action Input: JSON args
Observation: tool result
... (repeat)
Final Answer: concise answer with evidence
```

```python
from langchain import hub
from langchain.agents import create_react_agent, AgentExecutor
from langchain_openai import ChatOpenAI
from langchain.tools import Tool

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
prompt = hub.pull("hwchase17/react")  # ✅              prompt

tools = [
    Tool(name="search", func=search_web, description="       "),
    Tool(name="python", func=run_python, description="    Python    "),
]

agent = create_react_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, max_iterations=8)
print(executor.invoke({"input": "    A     B       "}))
```

-
-
-
-

- `max_iterations`
-
-
-
-

```
{
  "answer": "    ...",
  "evidence": ["url1", "url2"],
  "assumptions": ["    1"],
  "next_actions": ["       "]
}
```

②

- ____
- _____

`Thought/Action/Observation`

③

- _____
- _____

-
-
-

-
-

-
-
-

	→ →	

```
Actor
   ↓
Evaluator        /
   ↓(    )
Self-Reflection
   ↓
Memory Bank
   ↓
                  Prompt
```

-
-
-

```python
def solve_with_reflexion(task, memory_bank):
    hint = memory_bank.retrieve(task)
    result = actor(task, hint)
    score = evaluator(result)

    if score < 0.8:
        lesson = reflector(task, result, score)
        memory_bank.add(task, lesson)
        return solve_with_reflexion(task, memory_bank)

    return result
```

```
{
  "task_type": "financial_analysis",
  "failure_pattern": "used outdated source",
  "lesson": "must cite source date <= 30 days",
  "trigger": ["market size", "forecast"],
  "created_at": "2026-02-27"
}
```

-
-
-
-

-
-
-
-

④

- [illegible]
- [illegible]

-
-
-
-

```
User Goal
  ↓
Planner →
  ↓
Executor →
  ↓
Verifier →
  ↓
Re-planner →
```

```
- step_id
- objective
- required_tool
- success_criteria
- dependency
```

```
     step    +

1)
2)
3)
```


-
-
-
-

-
-
-
-

```yaml
goal:
constraints:
  budget_usd: 2
  deadline_min: 8
steps:
  - collect_data
  - clean_data
  - compute_metrics
  - generate_slides
  - qa_check
```

⑤

- _____
- ____ ____ _____

```
{
  "tool": "search_news",
  "arguments": {
    "query": "AIGC     2026",
    "top_k": 5
  }
}
```

```
{
  "name": "get_weather",
  "description": "           ",
  "parameters": {
    "type": "object",
    "properties": {
      "city": {"type": "string"},
      "unit": {"type": "string", "enum": ["c", "f"]}
    },
    "required": ["city"]
  }
}
```

-
-
-
-
-

-
-
-
-

-
-
-
-
-

⑥

- [illegible]
- [illegible]
- [illegible]

```
get_stock_price(symbol, date)
```


-
-
-

⑦

- _____
- _____
- _____

⑧

- _____
- _____

-
-
-
-
-

[illegible]

📚 [illegible]

- [illegible] → [illegible]
- [illegible] → [illegible]

→

- $\approx$
- $\approx$


```yaml
task:
  id: "fix-auth-bypass"
  graders:
    - type: deterministic_tests
      required: [test_empty_pw.py, test_null_pw.py]
    - type: llm_rubric
      rubric: prompts/code_quality.md
    - type: static_analysis
      commands: [ruff, mypy, bandit]
    - type: tool_calls
      required:
        - {tool: read_file}
        - {tool: edit_file}
        - {tool: run_tests}
```


```
1.            (  Eval Task)
2.    Eval (       )
3.    Agent (Prompt/Tool/   )
4.       Eval (       )
5.    Eval    →    Eval
6.
```


```yaml
eval_suite: market_research_agent
tasks:
  - id: "task_1"
    input: "..."
    graders: [...]
    success_criteria: "..."
```

```
User Query
  ↓
Planner
  ↓
Research Agent     /
  ↓
Analysis Agent     /
  ↓
Writer Agent
  ↓
Verifier
```

-
-
  -
  -
-

	⩾
	⩽
	⩽

			⩾
			⩾

-
-
-
-

- [illegible]
- [illegible]
- [illegible]
- [illegible]

- [illegible]
- [illegible]
- [illegible]
- [illegible]

- [illegible]
- [illegible]
- [illegible]
- [illegible]
- [illegible]
- [illegible]
- [illegible]
- [illegible]

- [illegible]
- [illegible]

- [illegible]
- [illegible]

- [illegible]
- [illegible]

-
-
-